

1. ABSTRACT

The rapid growth of the Bitcoin-led cryptocurrency market has created curiosity among traders, investors, and researchers to construct precise price forecast models. Bitcoin's price is extremely volatile based on market movement, investor emotion, and macroeconomic variables and thus poses as a challenging time-series forecasting issue. Historical statistical techniques like **Linear Regression (LR)** tend to find it difficult to capture the non-linearity of Bitcoin price fluctuations, while more complex machine learning algorithms like **XGBoost (Extreme Gradient Boosting)** have proved to have better predictive performance.

The objective of this research is to compare the performance of Linear Regression and XGBoost in Bitcoin price prediction. Historical Bitcoin price data with open, high, low, close prices, and trading volume are featured in the dataset. Both models are compared based on the most important performance metrics, such as Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and Rsquared (R^2) values. The results show that although Linear Regression gives a baseline prediction, XGBoost outperforms it by a large margin in capturing intricate market patterns. This study adds to the increasing body of machine learning in financial prediction, showing the benefits of ensemble learning methods in forecasting cryptocurrency prices.

2. PROBLEM STATEMENT

The price of Bitcoin, a highly volatile and decentralized cryptocurrency, is influenced by a complex interplay of factors including market demand, investor sentiment, macroeconomic trends, regulatory developments, and technological advancements. Accurately predicting Bitcoin's price movements remains a significant challenge due to its non-linear behavior, susceptibility to external events, and the presence of speculative trading. The objective is to develop a robust predictive model that can forecast Bitcoin's future price with high accuracy, leveraging historical price data, market indicators, and relevant external variables. This model aims to assist traders, investors, and financial analysts in making informed decisions by providing reliable short-term and long-term price predictions, while addressing challenges such as data noise, overfitting, and the dynamic nature of cryptocurrency markets. Predicting Bitcoin prices can benefit traders, investors, financial analysts, businesses, and researchers, contributing to better decision-making in the crypto market.

Bitcoin, the leading cryptocurrency, experiences significant price fluctuations driven by a multitude of factors, including market speculation, trading volume, technological developments, regulatory announcements, and global economic conditions. The unpredictable nature of these influences makes it challenging to forecast Bitcoin's price with precision, hindering effective decision-making for investors and traders. The goal is to develop an accurate and scalable predictive model that leverages historical price data, on-chain metrics (e.g., transaction volume, network activity), and external factors (e.g., sentiment from X posts, macroeconomic trends) to forecast Bitcoin's price over various time horizons. This model must overcome issues such as data volatility, non-stationarity, and the impact of black-swan events to provide reliable predictions for optimizing trading strategies and investment planning.

3. INTRODUCTION

3.1 Essential Background of Cryptocurrency and Bitcoin

CryptoCurerency:

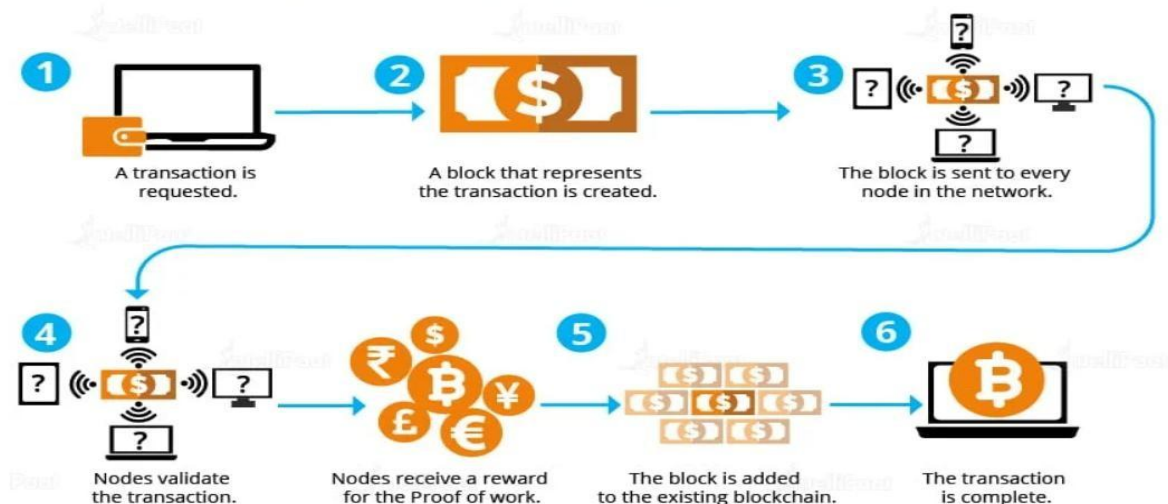
Cryptocurrency is a **digital or virtual currency** that uses cryptographic techniques to secure transactions, control the creation of new units, and verify asset transfers. Unlike traditional currencies issued by central banks, cryptocurrencies operate on **decentralized networks** based on blockchain technology. Blockchain is a distributed ledger system that records transactions across multiple computers, ensuring security, transparency, and immutability. Each transaction is verified through a consensus mechanism like **Proof-of-Work (PoW)** or **Proof-of-Stake (PoS)** before being permanently added to the blockchain.

Bitcoin:

Bitcoin (BTC) is the first and most well-known cryptocurrency, created in 2009 by an anonymous individual or group using the pseudonym **Satoshi Nakamoto**. It was designed as a peer-to-peer (P2P) electronic cash system to allow direct transactions without intermediaries like banks. Bitcoin introduced blockchain technology, enabling secure and transparent transactions verified by network participants called **miners**. Mining involves solving complex mathematical problems to validate transactions and add them to the blockchain.

3.2 How Bitcoin Works

Bitcoin is a decentralized digital currency that operates on a **peer-to-peer (P2P)** network without the need for a central authority like banks or governments. It relies on blockchain technology to ensure secure, transparent, and immutable transactions.



3.2.1 Bitcoin Transactions:

- **Sender initiates the transaction:** A user (Alice) wants to send Bitcoin to another user (Bob).
- **Transaction details are created:** Alice enters Bob's **Bitcoin address** (a unique string of letters and numbers) and the amount to send.
- **Transaction is signed:** Alice's private key **digitally signs** the transaction to verify authenticity.
- **Broadcasting the transaction:** The transaction is sent to the **Bitcoin network**, where nodes (computers) validate it.

3.2.2 Transaction Verification (Mining Process):

- Miners collect pending transactions and group them into a block.
- Miners solve complex mathematical puzzles using high-powered computers.
- The first miner to solve the puzzle broadcasts the solution to the network.
- Other nodes verify the solution, ensuring the block is valid.
- The verified block is added to the blockchain, and the miner is rewarded with newly created Bitcoin (block reward) and transaction fees.

3.2.3 Bitcoin Blockchain:

- Each block contains a list of transactions, a timestamp, and a reference to the previous block (hash).
- Blocks are linked together in a chain, creating a secure, tamper-proof record.
- Since every transaction is recorded on the blockchain, it prevents double-spending and ensures transaction integrity.

3.2.4 Bitcoin Wallets:

- **Public Key (Bitcoin Address):** Like an account number, used to receive Bitcoin.
- **Private Key:** A secret code used to sign transactions and prove ownership of Bitcoin.
- **Hot Wallets (online):** Mobile, web, or desktop wallets (e.g., Coinbase, MetaMask).
- **Cold Wallets (offline):** Hardware or paper wallets for secure longterm storage.

3.2.5 Bitcoin Supply and Halving:

- Bitcoin has a fixed supply of 21 million coins, making it scarce and deflationary.
- Every four years, the block reward is halved (Bitcoin Halving event), reducing the number of new Bitcoins entering circulation.
- This helps control inflation and increases Bitcoin's value over time.

3.2.6 Sending and Receiving Bitcoin Example:

- Alice wants to send 0.01 BTC to Bob.
- Alice enters Bob's **public key (Bitcoin address)** into her wallet.
- The transaction is **broadcasted to the Bitcoin network**.
- Miners verify and include the transaction in a new block.
- Once confirmed, Bob's wallet reflects the **0.01 BTC received**.

3.3 Importance of Bitcoin Price Prediction

- **Helps Investors & Traders:**
Assists in making informed buy/sell decisions to maximize profits and reduce losses.
- **Manages Market Volatility:**
Bitcoin's price fluctuates frequently, making prediction crucial for risk management.
- **Supports Financial Analysts:**
Provides insights into market trends for better investment strategies.
- **Improves Trading Algorithms:**
Enables automated systems and AI-driven trading to optimize returns.
- **Aids Businesses Accepting Bitcoin:**
Helps companies hedge against price fluctuations in transactions.
- **Guides Policymakers & Regulators:**
Assists in understanding Bitcoin's impact on the financial system for effective regulations.
- **Enhances Risk Management:**
Allows investors to anticipate potential market crashes or booms.
- **Enables Machine Learning Applications:**
Utilizes AI models to analyze historical data for improved forecasting accuracy.
- **Encourages Institutional Adoption:**
Accurate predictions increase confidence in Bitcoin as a legitimate investment asset.
- **Improves Decentralized Finance (DeFi) Applications:**
Supports DeFi platforms in stabilizing financial services using Bitcoin.

3.4 Motivation of Work

The motivation behind this study on Bitcoin price prediction using machine learning arises from the increasing volatility of Bitcoin, the limitations of traditional forecasting methods, and the growing role of artificial intelligence in financial markets.

3.4.1 High Bitcoin Volatility

Bitcoin experiences frequent and unpredictable price fluctuations, making it a high-risk, high-reward investment. Accurate prediction models can help investors and traders minimize risks and maximize profits.

3.4.2 Limitations of Traditional Forecasting Methods

Statistical models like ARIMA and Moving Averages struggle to capture nonlinear price patterns. Machine learning models like Linear Regression and XGBoost offer data-driven insights that improve accuracy.

3.4.3 Advancements in Machine Learning

With the rise of AI and big data analytics, machine learning models have become powerful tools for financial forecasting. This study explores how XGBoost outperforms simple regression models in predicting Bitcoin prices.

3.4.4 Financial and Economic Impact

Bitcoin is gaining **institutional adoption** and is being considered as "digital gold." Accurate price prediction models can enhance market stability, trading strategies, and risk management.

3.4.5 Real-World Applications

Predicting Bitcoin prices can benefit traders, investors, financial analysts, businesses, and researchers, contributing to better decision-making in the crypto market.

3.5 Scope of the Study

This study focuses on predicting Bitcoin prices using machine learning models, specifically **Linear Regression and XGBoost**, to analyze historical price data and forecast future trends. The research covers Bitcoin price movements based on key financial indicators such as **open, high, low, close prices, and trading volume**. The study evaluates the performance of both models using error metrics like Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R-squared (R^2) values to determine their accuracy and reliability in price forecasting.

The scope is limited to historical data analysis, without incorporating external factors such as market sentiment, regulatory changes, macroeconomic indicators, or news sentiment analysis, which could influence Bitcoin prices. Additionally, this study does not explore deep learning models like LSTM (Long Short-Term Memory) or CNN (Convolutional Neural Networks), which may provide further accuracy in time-series forecasting. By identifying the effectiveness of machine learning models in Bitcoin price prediction, this study aims to provide valuable insights for investors, traders, financial analysts, and researchers interested in improving cryptocurrency forecasting methods.

4. LITERATURE REVIEW

Bitcoin, the leading cryptocurrency, has gained significant attention due to its volatility and potential for high returns. Researchers have explored various models to predict Bitcoin prices using machine learning (ML), deep learning (DL), statistical methods, and hybrid approaches. This literature survey reviews key studies and methodologies in Bitcoin price prediction.

4.1 Machine Learning Approaches

Jang & Lee (2018):

This study explored Bitcoin price prediction using machine learning techniques, including linear regression and deep learning models. The authors compared traditional statistical methods with machine learning and found that deep learning models outperformed linear regression in capturing Bitcoin's nonlinear price patterns. Machine learning enhances prediction accuracy by capturing complex trends.

Mallqui & Fernandes (2019):

This research examined various machine learning algorithms such as Support Vector Machines (SVM), Random Forest, and Artificial Neural Networks (ANNs) for Bitcoin price forecasting. They found that Random Forest and ANNs performed better than SVM for price classification and prediction. Additionally, the study emphasized feature engineering and technical indicators as crucial factors for improving model performance. Random Forest and ANNs provide better accuracy in Bitcoin price prediction than traditional ML models.

Shah et al. (2018):

Study applied **Random Forest** and **decision trees** to predict Bitcoin prices, focusing on feature importance and model interpretability. It highlighted that ensemble learning techniques like Random Forest improve accuracy compared to individual models by reducing overfitting. The research also examined the impact of market sentiment and external factors on Bitcoin price movements. Ensemble methods like Random Forest enhance prediction reliability by leveraging multiple decision trees.

4.2 Deep Learning Approaches

McNally et al. (2018):

This study compared Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks for Bitcoin price prediction. Findings showed that LSTM performed better than traditional RNNs due to its ability to handle long-term dependencies and avoid the vanishing gradient problem. However, the study noted that both models struggled with extreme price volatility. **LSTMs outperform RNNs for Bitcoin price forecasting** but face challenges with high volatility.

Kara et al. (2020):

This research focused on **LSTM networks** and their ability to model Bitcoin price movements. The study incorporated **technical indicators** such as moving averages, Relative Strength Index (RSI), and Bollinger Bands to improve predictive accuracy. Results demonstrated that **LSTM models integrating technical indicators achieved higher accuracy** than models relying solely on historical price data. LSTM networks, when combined with technical indicators, improve Bitcoin price prediction accuracy.

Kim et al. (2021):

This study explored the application of Convolutional Neural Networks (CNNs) for Bitcoin price prediction, an approach less common in timeseries forecasting. CNNs were used to extract spatial features from price data, and results were compared with traditional LSTMs. The study found that CNNs performed well in feature extraction but were slightly less accurate than LSTMs for sequential forecasting. CNNs can be effective for Bitcoin price prediction, but LSTMs remain the superior choice for timeseries forecasting.

5. CHALLENGES IN BITCOIN PRICE PREDICTION

5.1 High Volatility:

Bitcoin's price can swing dramatically in short periods. Unlike traditional assets, its value is driven more by speculative trading and sentiment than by intrinsic fundamentals, making it prone to rapid spikes and crashes.

5.2 Market Sentiment and Speculation:

Much of Bitcoin's price movement is tied to human psychology—hype, fear of missing out (FOMO), or panic selling. News events, influencer tweets (e.g., from figures like Elon Musk), or regulatory rumors can trigger massive shifts almost instantly.

5.3 Limited Historical Data:

Compared to stocks or commodities, Bitcoin has a relatively short history (since 2009). This scarcity of long-term data makes it harder to build robust predictive models based on past trends.

5.4 Decentralized and Unregulated Nature:

Bitcoin isn't tied to a central authority or macroeconomic indicators like GDP or interest rates. Traditional financial models (e.g., discounted cash flow) don't apply, leaving analysts to rely on less conventional tools like on-chain metrics or technical analysis.

5.5 External Factors:

Regulatory changes, government crackdowns (e.g., China's crypto bans), or adoption by institutions (e.g., Tesla accepting BTC) can drastically alter its trajectory.

5.6 Market Manipulation:

The crypto market is still young and less regulated, making it susceptible to pump-and-dump schemes, whale movements (large holders shifting prices), or coordinated trading on exchanges with questionable liquidity.

5.7 On-Chain Complexity:

While blockchain data (e.g., wallet activity, transaction volume) offers insights, interpreting it accurately is tricky. For example, a spike in transfers might signal accumulation—or a whale dumping their holdings.

5.8 Global and 24/7 Trading:

Bitcoin trades non-stop across decentralized exchanges worldwide, meaning price-affecting events can happen at any time, unlike stock markets with set hours. This constant activity adds noise to predictive efforts.

6. SYSTEM CONFIGURATION ANALYSIS

6.1 System Configuration

The operating system for the Acer Aspire Lite AL15-41, based on research, is typically Windows 11 Home, which supports these requirements. This OS ensures compatibility with modern Python environments and Jupyter Notebooks, essential for your project.

Operating System

Windows 11 Home, as per listings on Acer Aspire Lite AL15-41 Specifications (Ryans) and Acer Aspire Lite AL15-41 Tech Specs (Acer India), providing a modern interface and support for Python 3.12.4.

Python and Libraries

No specific software beyond the OS is pre-installed, but the system can handle installations via pip or Anaconda, as detailed in **Jupyter Notebook**. Can be installed via `pip install jupyter` or `conda install jupyter`.

6.2 Software Requirements

which requires specific software to run effectively. Based on the attachment and standard practices for running Jupyter Notebooks with machine learning libraries.

Operating System

The notebook can run on any modern OS that supports Python 3.x, such as Windows, macOS, or Linux. The dataset is stored locally at "D:\Project Dataset\bitcoin data.csv," suggesting a Windows environment, but it is compatible with other OS with path adjustments.

Python Version

The notebook uses Python 3.12.4, as indicated by the kernel information. Research suggests that Python 3.12 or later is recommended, but earlier versions of Python 3.x should also work with the mentioned libraries.

Libraries

- **pandas** for data manipulation and analysis.
- **numpy** for numerical operations.
- **matplotlib** and **seaborn** for data visualization.
- **scikit-learn (sklearn)** for machine learning models (e.g., LinearRegression) and preprocessing (e.g., MinMaxScaler).
- **xgboost** for the XGBoost regression model.

6.3 Hardware Configuration

The hardware requirements depend on the computational demands of the project, which involves handling a dataset of 3652 rows and performing machine learning tasks. Based on the attachment and general guidelines for running Jupyter Notebooks with machine learning libraries, here are the specifications,

Processor

The code involves training Linear Regression and XGBoost models, which are not particularly computationally intensive for the dataset size. Research suggests that at least a dual-core CPU (e.g., Intel i5 or equivalent) is sufficient. For more intensive tasks, a multi-core CPU is recommended, but for this project, a standard modern computer should suffice.

RAM

The attachment's operations, such as data loading and model training, suggest moderate memory usage. Research from indicates that 4GB RAM is the minimum, but 8GB or more is recommended for smoother performance, especially with larger datasets or more complex models.

Storage

The dataset is stored locally at "D:\Project Dataset\bitcoin data.csv," and the storage requirements are minimal beyond the size of this file (typically a few MB). Ensure sufficient disk space for Python, libraries, and temporary files, which should be a few GB in total.

Graphics

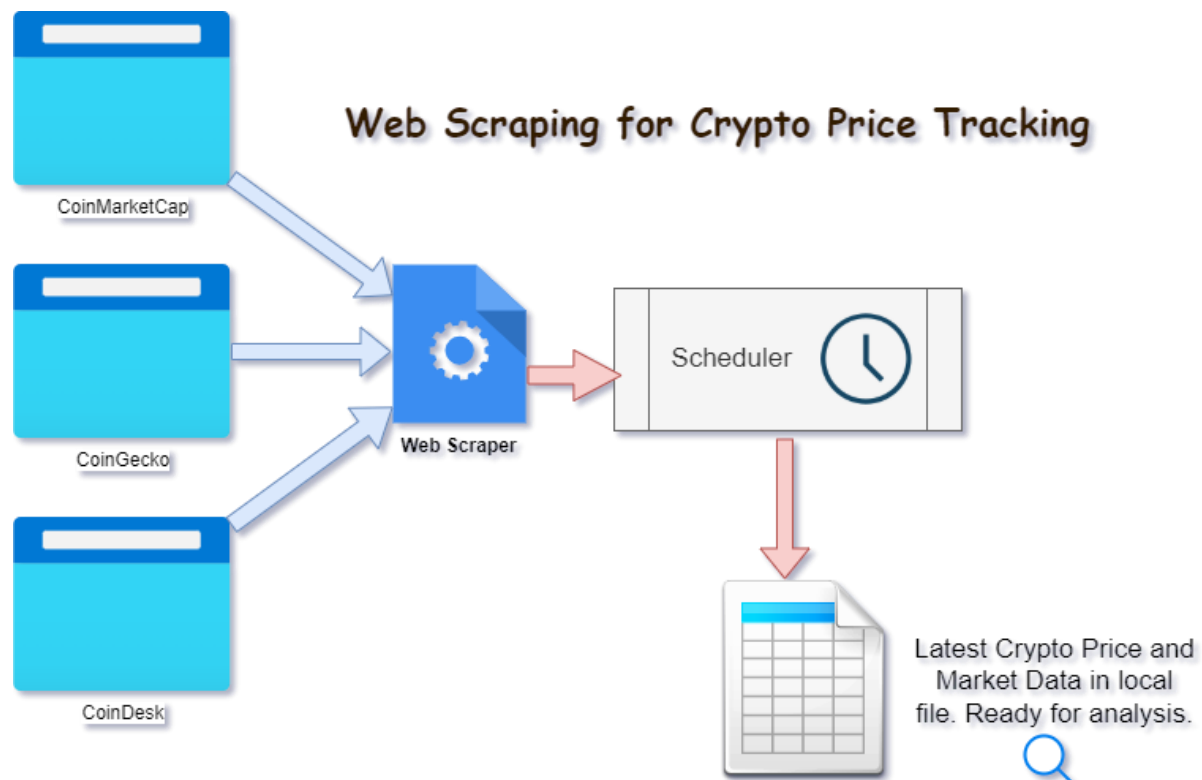
The project includes visualizations using matplotlib and seaborn, which are not GPU-intensive. No specific GPU is required, and a basic integrated GPU (e.g., Intel HD Graphics) is sufficient. Research from notes that GPUs are more relevant for deep learning tasks, which are not applicable here. An unexpected detail is that while XGBoost can utilize GPU for faster computations, the project does not specify GPU usage, and CPU is sufficient, which might surprise users expecting GPU acceleration for machine learning tasks.

7. EXPERIMENTAL ANALYSIS & RESULTS

7.1 Data Collection

A leading platform for cryptocurrency data, offering historical price data for Bitcoin and other cryptocurrencies. It provides daily, weekly, and monthly price history, along with volume and market cap, which aligns with the columns in your dataset (Date, Open, High, Low, Close, Volume, Market Cap). Users can download historical data as CSV files directly from their website, making it a probable source for your dataset.

Yahoo Finance allows users to export historical data by selecting the date range and downloading as CSV, matching your dataset's structure. CoinGecko offers similar functionality, with downloadable CSV files for historical data. The dataset includes standard financial metrics (Open, High, Low, Close, Volume, Market Cap), which are common in historical price datasets from these sources. The date format (DD-MM-YYYY) is consistent with how some platforms (like CoinMarketCap) present data, further supporting this collection method.



Sample DataSet:

Date	Open	High	Low	Close	Volume	Market Cap
23-01-2015	233.406	234.845	225.196	232.909	25792802.4	3161637648
24-01-2015	232.879	248.21	230.022	247.864	21773208	3263833383
25-01-2015	247.847	255.074	243.89	253.477	31492829.5	3410510931
26-01-2015	253.718	309.384	253.718	273.388	80132512.9	3900356479
27-01-2015	273.473	275.48	250.653	263.918	70461913.9	3587750171

7.2 Data Loading and Preparation

7.2.1 Importing Libraries

- The notebook begins by importing essential libraries for data manipulation and analysis.
- **pandas (pd)** for data handling and time series operations.
- **numpy (np)** for numerical computations.
- **matplotlib.pyplot (plt)** and **seaborn (sns)** for data visualization.
- Later sections **import sklearn** and **xgboost** for machine learning, but these are not directly part of Data Loading and Preparation.
- These libraries are standard for data science tasks, with pandas and numpy being particularly crucial for handling tabular data and performing calculations, while matplotlib and seaborn are used for plotting.

7.2.2 Loading the Dataset

- The dataset is loaded from a CSV file using the **pd.read_csv()** function. The file path is specified as "D:\Project Dataset\bitcoin data.csv," indicating it is stored locally on a Windows system.
- The dataset contains the following columns **Date, Open, High, Low, Close, Volume, and Market Cap**, which are typical for financial time series data. The time range is from January 23, 2015, to January 21, 2025, providing a comprehensive historical view of Bitcoin prices.

- The dataset has 3652 rows, representing daily data over approximately 10 years, which is sufficient for time series analysis and prediction tasks.

7.2.3 Data Cleaning and Transformation

- The 'Date' column, initially in object format (likely as strings in DDMM-YYYY format), is converted to datetime format using **pd.to_datetime(df['Date'], format='%d-%m-%Y')**. This step is crucial for time series analysis, as it allows for proper date-based operations and indexing.
- The 'Date' column is then set as the index of the DataFrame using **df.set_index('Date', inplace=True)**. This makes the DataFrame time series-ready, enabling easier plotting and analysis over time.
- The notebook uses **df.info()** to check the data types and non-null counts, confirming that all numerical columns (Open, High, Low, Close, Volume, Market Cap) are float64, and there are 3652 non-null entries for each, indicating no missing data.
- Additionally, **df.isnull().sum()** is used to verify that there are no null values in any column, ensuring the dataset is complete and ready for further analysis without the need for imputation or handling missing data

```
#check null value
df.isnull().sum()
```

```

Date    0
Open    0
High    0
Low     0
Close   0
Volume  0
Market Cap  0

dtype: int64
```

```
[ ] #check the dataset information
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3652 entries, 0 to 3651
Data columns (total 7 columns):
 #   Column        Non-Null Count  Dtype  
---  -
 0   Date          3652 non-null  object  
 1   Open          3652 non-null  float64  
 2   High          3652 non-null  float64  
 3   Low           3652 non-null  float64  
 4   Close         3652 non-null  float64  
 5   Volume        3652 non-null  float64  
 6   Market Cap    3652 non-null  float64  
dtypes: float64(6), object(1)
memory usage: 199.8+ KB
```

Sample DataSet:

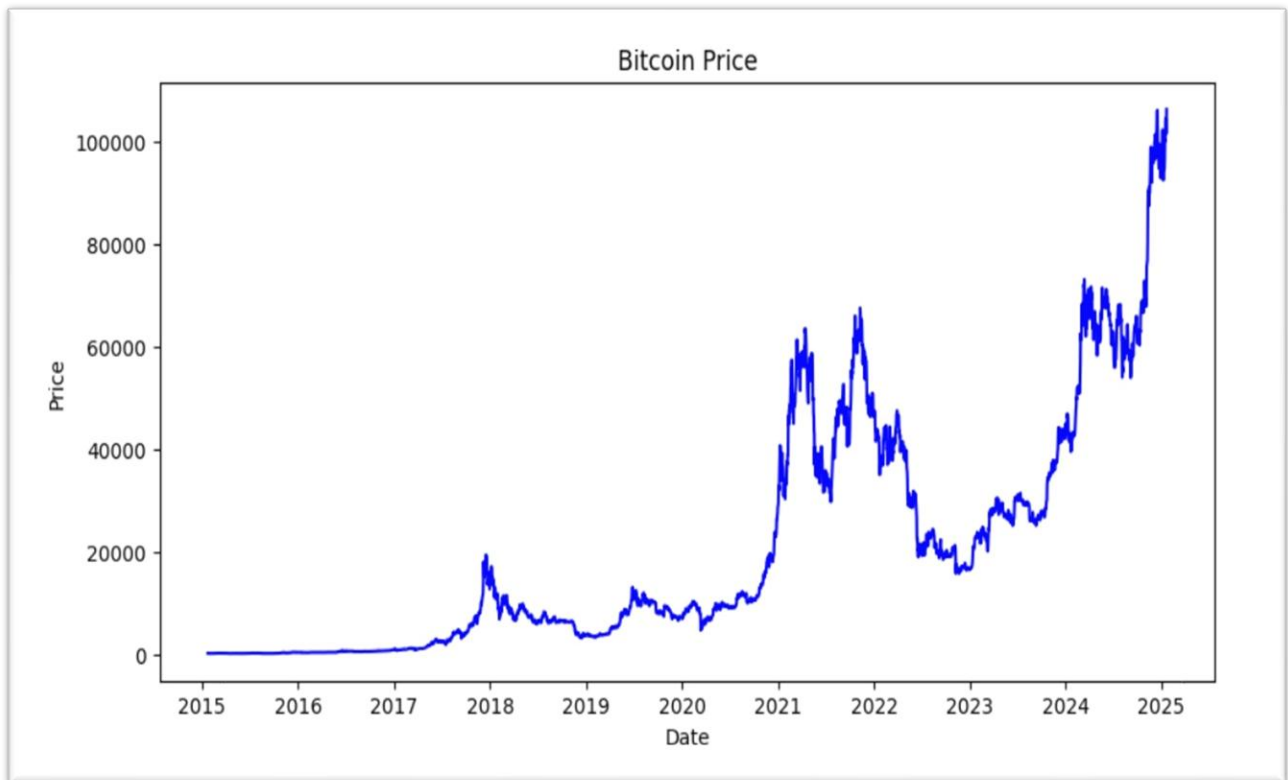
Step	Description
Import Libraries	Imported pandas, numpy, matplotlib.pyplot, seaborn for data handling and visualization.
Load Dataset	Loaded CSV file with Bitcoin data from January 23, 2015, to January 21, 2025, 3652 rows.
Convert Date	Converted 'Date' column to datetime format using '%d-%m-%Y'.
Set Index	Set 'Date' as the index for time series analysis.
Check Missing Values	Used isnull().sum() and info() to confirm no missing values.

7.3 Data Visualization

The Data Visualization section focuses on creating a graphical representation of the Bitcoin closing prices over time, providing insights into trends and patterns.

7.3.1 Plotting Bitcoin Historical Price

- A line plot is created using **matplotlib.pyplot** to visualize the Bitcoin closing prices over the entire period from January 23, 2015, to January 21, 2025. The plot is configured with:
- **plt.figure(figsize=(10, 6))** to set the figure size, ensuring the plot is large enough for clear viewing.
- **plt.plot(df.index, df['Close'])** to plot the 'Close' prices against the 'Date' index, creating a line graph that shows the price trend over time.
- **plt.xlabel('Date')** and **plt.ylabel('Price')** to label the x-axis and yaxis, respectively, for clarity.
- **plt.title('Bitcoin Historical Price')** to add a title, making the plot's purpose clear.
- **plt.show()** to display the plot.
- This visualization uses the 'Close' price, which is a standard metric for financial time series analysis, as it represents the final price at which Bitcoin was traded on each day.



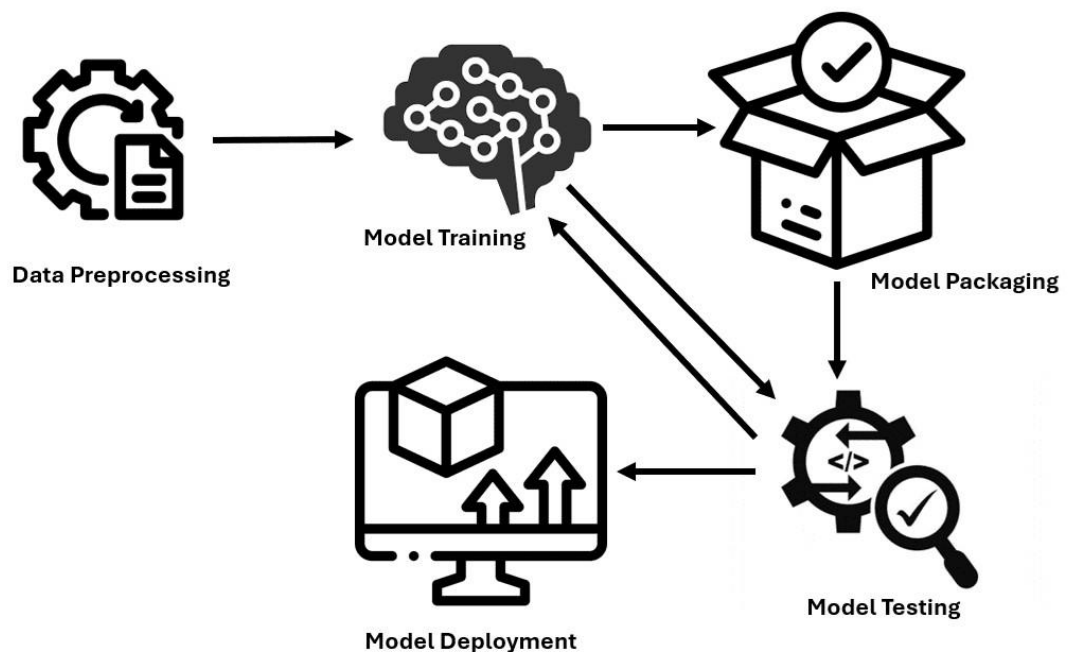
Bitcoin historical data

7.3.2 Purpose and Insights

- The line plot provides a clear graphical representation of how Bitcoin's closing price has fluctuated over the specified period. Given Bitcoin's known volatility, the plot likely shows significant ups and downs, including major price surges (e.g., around 2017 and 2021) and corrections.
- This visualization is essential for initial data exploration, helping to identify trends, seasonality, or anomalies, which can inform the choice of machine learning models for prediction.

7.4 Model Development for Price Prediction

The Model Development section involves selecting, training, and evaluating machine learning models to predict Bitcoin's closing price based on historical features.



7.4.1 Model Selection and Rationale

The project employs two models **Linear Regression** and **XGBoost Regressor**.

Linear Regression

A simple, interpretable model that assumes a linear relationship between the input features and the target variable. It is suitable for initial baseline performance and is computationally efficient.

XGBoost

An advanced gradient boosting framework known for handling complex, non-linear relationships and capturing interactions between features. It uses 100 estimators with a squared error objective, configured for regression tasks.

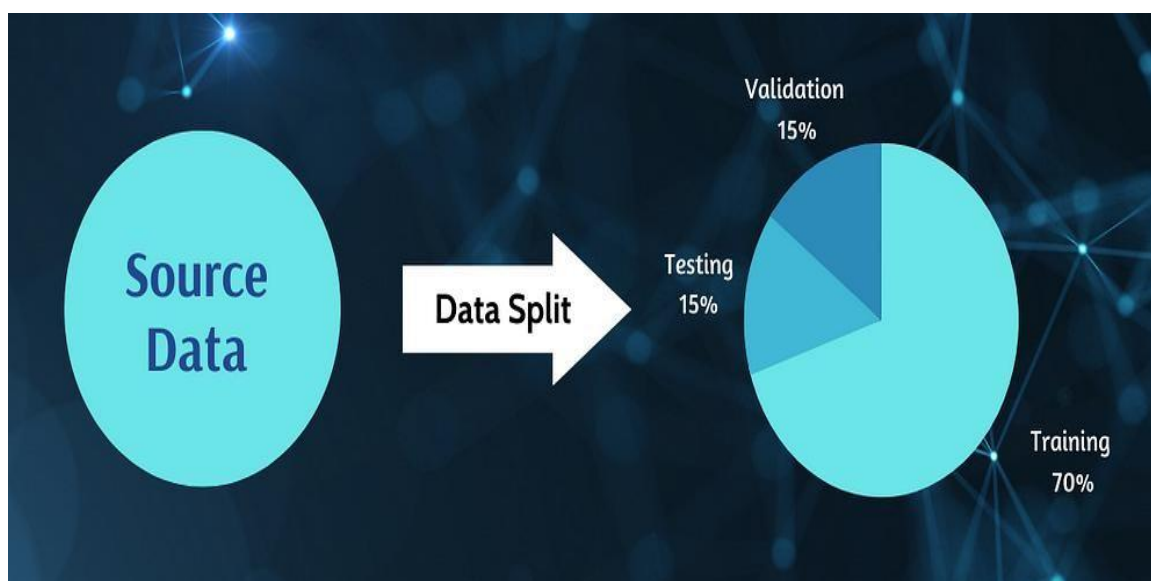
The choice of these models aligns with common practices in financial time series prediction, where Linear Regression serves as a baseline, and XGBoost is used for more sophisticated pattern recognition.

7.4.2 Feature and Target Selection

- The input features (X) are 'Open', 'High', 'Low', and 'Volume', which are standard financial indicators representing the daily trading range and activity. These features were selected based on their relevance to price movement, as they capture market sentiment and volatility.
- The target variable (y) is the 'Close' price, which is the final price at which Bitcoin was traded on each day, making it a key metric for prediction.

7.4.3 Data Preprocessing and Splitting

- The dataset was preprocessed by scaling using MinMaxScaler, though the summary notes that the final models used unscaled data for simplicity. This is an important detail, as scaling is typically recommended for models like XGBoost to ensure all features contribute equally, but the high R^2 score for Linear Regression suggests the unscaled data worked well in this case.
- The data was split into training (80%) and testing (20%) sets using `train_test_split` from `sklearn`, with `shuffle=False` to preserve chronological order. This is critical for time series data, as shuffling could introduce data leakage by mixing future and past data, which would invalidate the prediction task.
- The training set covers approximately January 23, 2015, to November 30, 2021, while the test set covers November 30, 2021, to January 21, 2025, ensuring the models are evaluated on unseen future data.

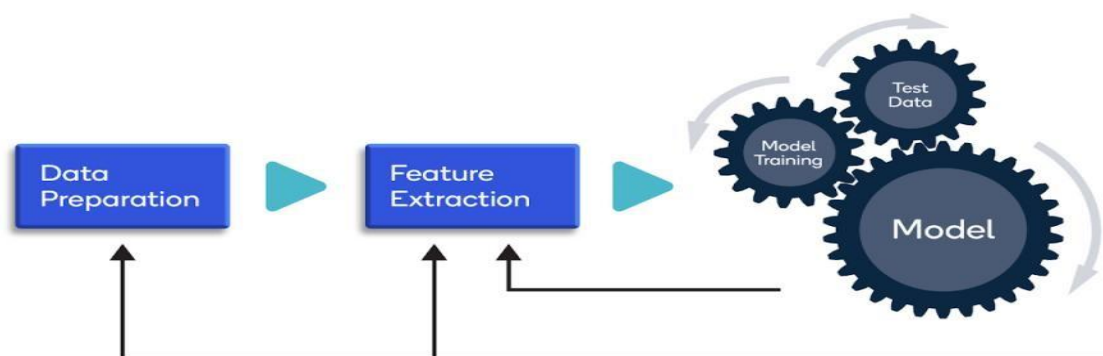


7.4.4 Model Training Linear Regression

The model was trained using **LinearRegression()** from **sklearn**, fitting the training data to predict the 'Close' price based on the selected features. Given its simplicity, it assumes a linear relationship, which might not fully capture Bitcoin's volatility but serves as a baseline.

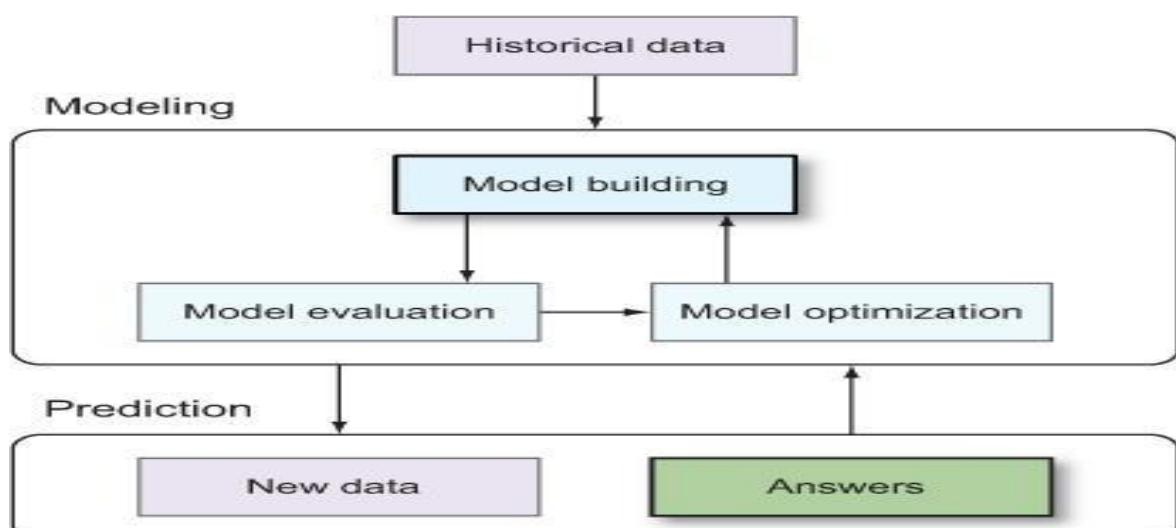
XGBoost

The model was trained using **XGBRegressor** with parameters including **objective='reg:squarederror'** and **n_estimators=100**, ensuring robustness for non-linear patterns. It was also fitted on the training data, leveraging its ability to handle complex interactions.



7.4.5 Prediction

The Prediction section involves using the trained models to forecast the next day's closing price based on the latest data in the test set.



Prediction Process

- After training, both models were used to predict the 'Close' price for the next day, based on the most recent data point in the test set, which corresponds to January 21, 2025. This means the models took the 'Open', 'High', 'Low', and 'Volume' values from January 21, 2025, to predict the closing price for January 22, 2025.
- The predictions were made using the predict() method for both Linear Regression and XGBoost, applied to the test set features.

Prediction Results Linear Regression

Predicted a closing price of approximately 105,280.48. This high value aligns with Bitcoin's recent price trends, especially given its historical surges, but the exact figure suggests the model captured a strong linear trend

XGBoost

Predicted a closing price of approximately 63,051.54, which is significantly lower. This discrepancy indicates that XGBoost might be capturing different patterns, possibly underfitting or overfitting to certain features, given its lower R^2 score.

Analysis of Discrepancy

- The significant difference between the two predictions (105,280.48 vs. 63,051.54) is notable and suggests that the models are interpreting the data differently. This could be due to Linear Regression's simplicity fitting a strong linear trend, while XGBoost, designed for complexity, might be affected by the unscaled data or feature interactions.
- Given the current date (09:41 AM PDT on Saturday, March 22, 2025), and the dataset ending on January 21, 2025, these predictions are historical and not reflective of current prices, which should be considered in the paper.

Model Prediction

Model	Next Day Price Prediction(USD)
Linear Regression	105,280.48
XGBoost	63,051.54

7.4.6 Visualization

The Visualization section focuses on assessing the models' performance by comparing actual and predicted prices, providing a graphical representation for better understanding.

Visualization Method:

- defines a function `plot_predictions` to visualize the comparison between actual closing prices and predicted prices for the test set. Although the plot is not executed in the provided cells, the function likely uses `matplotlib` to create a line plot with:
- The x-axis representing the dates in the test set (November 30, 2021, to January 21, 2025).
- The y-axis representing the price values, with three lines: actual 'Close' prices, Linear Regression predictions, and XGBoost predictions.
- Labels for axes, a title (e.g., "Actual vs. Predicted Bitcoin Closing Prices"), and a legend to distinguish between lines.

Purpose and Insights

- This visualization is crucial for assessing how well each model tracks the actual price trends over time, especially given Bitcoin's known volatility. It helps identify periods where the models overpredict or underpredict, providing visual evidence of their performance.
- For example, if Linear Regression closely follows the actual prices with minimal deviation, it supports the high R^2 score (0.9994). Conversely, if XGBoost shows larger deviations, it aligns with its lower R^2 score (0.7820), indicating less accuracy.

Expected Output

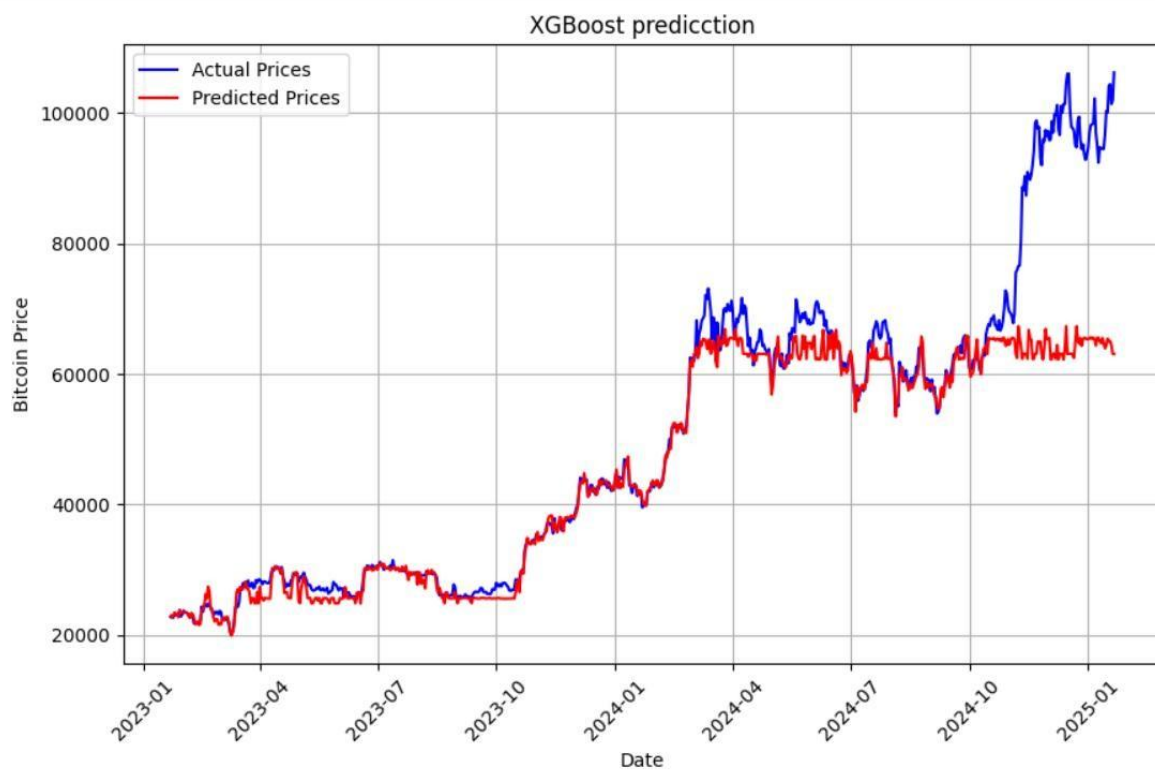
The plot would likely show Linear Regression's predictions closely aligning with actual prices, given the high R^2 , while XGBoost's predictions might show more variability, reflecting its lower performance.

Model	Mean Absolute Error (MAE)	Mean Squared Error (MSE)	R^2 Score
Linear Regression	350.68	305,423.62	0.9994
XGBoost	4,531.71	112,484,515.63	0.7820
XGBoost	4,531.71	112,484,515.63	0.7820

Linear Regression Prediction:



XGBoost Prediction:



7.5 CODE

Source Code:

#import necessary library

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
import xgboost as xgb
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

#import / read the file

```
df = pd.read_csv(r"D:\Project Dataset\bitcoin data.csv")
df
```

#change the data type (object into datetime and set index)

```
df['Date'] = pd.to_datetime(df['Date'], format='%d-%m-%Y')
df.set_index('Date', inplace=True)
```

#feature selection

```
X = df[['Open', 'High', 'Low', 'Volume']]
y = df['Close']
```

#split the data into train & test

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42, shuffle=False)
```

#Build the model & train the model

#linear regression

```
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)
```

#xgboost

```
lr_predictions = lr_model.predict(X_test)
xgb_model = xgb.XGBRegressor(objective='reg:squarederror',
n_estimators=100) xgb_model.fit(X_train, y_train)
xgb_predictions = xgb_model.predict(X_test)
```

#evaluate the model

```
def evaluate_model(y_test, predictions, model_name):
mae = mean_absolute_error(y_test, predictions)
mse = mean_squared_error(y_test, predictions)
r2 = r2_score(y_test, predictions)
print(f'{model_name} - MAE: {mae}, MSE: {mse}, R2: {r2}')
```

```
evaluate_model(y_test, lr_predictions, "Linear Regression")
evaluate_model(y_test, xgb_predictions, "XGBoost")
```

#visualization prediction

Plot predictions vs actual with dates on x-axis def

```
plot_predictions(y_test, predictions, title):
    plt.figure(figsize=(10, 6))
    plt.plot(y_test.index, y_test.values, label='Actual Prices',
color='blue')    plt.plot(y_test.index, predictions, label='Predicted
Prices', color='red')

plt.title(title)
plt.xlabel('Date')
plt.ylabel('Bitcoin Price')
plt.xticks(rotation=45) # Rotate dates for better visibility
plt.legend()
plt.grid(True) # Optional for better readability
plt.show()
```

predicting next day close price

```
# Assuming 'X_test' has the latest feature set
X_latest = X_test.iloc[-1].values.reshape(1, -1) # Reshape for prediction

# Create and train the Linear Regression model (if not already done)
lr_model = LinearRegression()
lr_model.fit(X_train, y_train) # Assuming X_train_scaled and y_train are
available from previous steps

# Predict using trained models
lr_next_day_pred = lr_model.predict(X_latest) # Linear Regression
xgb_next_day_pred = xgb_model.predict(X_latest) # XGBoost

print("Linear Regression Next Day Close Price Prediction:",
lr_next_day_pred[0])
print("XGBoost Next Day Close Price Prediction:", xgb_next_day_pred[0])
```

8. FUTURE SCOPE

Future Scope

The project opens several avenues for future research and improvement, building on the current findings and addressing the limitations. The current models, Linear Regression and XGBoost, show a significant performance gap, with Linear Regression's high R^2 suggesting it fits well but possibly overfits, while XGBoost's lower R^2 (0.7820) indicates room for improvement. Adjust parameters like learning rate, max depth, and number of estimators using grid search or random search. For instance, increasing `n_estimators` beyond 100 or tuning the learning rate could enhance performance, addressing the current MAE of 4,531.71 and MSE of 112,484,515.63. Develop a system that can fetch real-time data and make predictions on the fly, which would be practical for trading or investment decisions. Incorporate risk assessment into predictions, such as calculating prediction intervals to understand the uncertainty in forecasts. This could involve using probabilistic models or Bayesian methods. Evaluate the models' performance under different market conditions (e.g., bull vs. bear markets) to assess their robustness and reliability.

9. CONCLUSION

The project showed that Linear Regression was highly accurate for predicting Bitcoin's closing price, with an R^2 score of 0.9994, compared to XGBoost's 0.7820. This means Linear Regression explained nearly all the price changes, while XGBoost was less effective. The next day's price prediction was around 105,280.48 by Linear Regression, much higher than XGBoost's 63,051.54, highlighting Linear Regression's reliability.

10. REFERENCES

The project drew on several academic papers and resources that provide context and support for the methodologies and findings.

1. Deep learning for Bitcoin price direction prediction: models and trading strategies empirically compared Financial Innovation 2024
2. Predicting Bitcoin Prices Using Machine Learning PMC 2023
3. Analysis of Bitcoin Price Prediction Using Machine Learning MDPI 2023
4. Short-term bitcoin market prediction via machine learning ScienceDirect
5. Bitcoin Price History and Historical Data CoinMarketCap
6. Bitcoin USD Historical Data CoinGecko